DO NOT REMOVE FROM EXAM VENUE

University of Plymouth

MODULE CODE:	**ELEC351**
TITLE OF PAPER:	**Advanced Embedded Programming**

TIME ALLOWED	**3 hours**
DATE	**Thursday 11th January 2024**
TIME	**09:00 AM – 12:00 PM**
FACULTY	**Science and Engineering**
SCHOOL	**Engineering, Computing and Mathematics**
ACADEMIC YEAR	**2023/2024**
STAGE	**FOUR**

INSTRUCTIONS TO CANDIDATES:

Answer **ALL** questions in **Section A** and **TWO** questions from **Section B**.

Marks for parts of questions are shown where appropriate. Remember to give justification for your answers.

Candidates are not permitted to look at the examination paper until instructed to do so.

SECTION A: Answer ALL questions.

Q1. Class Types

Consider the following code:

Listing 1: Listing for Section A, Q1

```
1  #include <iostream>
2  using namespace std;
3
4  class Complex {
5      private:
6      double real;
7      double imag;
8
9      public:
10     Complex(double r, double i) {
11         real = r;
12         imag = i;
13     }
14     double magnitude() {
15         return sqrt(real*real + imag*imag);
16     }
17 };
18
19 int main()
20 {
21     Complex c1(2.0,3.0);
22     cout << c1.magnitude() << endl;
23
24     //Update and recalculate
25     c1.imag = 0.0;
26     cout << c1.magnitude() << endl;
27     return 0;
28 }
```

(a) The code above does not build. Why is this?

[2 Marks]

(b) While maintaining best practise, what would you suggest needs to be done to correct this error? Justify your answer.

[3 Marks]

(Over/...)

Q2. Class Inheritance

The code below builds and runs without any known issues.

Listing 2: Listing for Section A, Q2

```
1  #include <iostream>
2  using namespace std;
3
4  class Number {
5      protected:
6      double real;
7
8      public:
9      Number(double u) {
10         real = u;
11     }
12     virtual void display() {
13         cout << real << endl;
14     }
15 };
16
17 class Complex : public Number {
18     private:
19     double imag;
20     public:
21     Complex(double r, double i) : Number(r) {
22         imag = i;
23     }
24     virtual void display() {
25         cout << real << "+" << "j" << imag << endl;
26     }
27 };
28
29 int main() {
30     Complex c1(2.0,3.0);
31     c1.display();
32
33     Number* c2 = new Complex(2.0,3.0);
34     c2->display();
35     delete c2; c2 = nullptr;
36
37     return 0;
38 }
```

(a) When `c1` is instantiated on line 30, the members `real` and `imag` are initialised. In what order is this done and why?

[3 Marks]

(Over/...)

(b) On line 17 of Listing 2, we can see the class `Complex` inherits the parent class `Number`, and this includes the use of the keyword `public`. Brieflt explain the meaning of the keyword `public` in this context?

[2 Marks]

(c) On line 33 of Listing 2 we see the following:

```
Number* c2 = new Complex(2.0,3.0);
```

The data type of `c2` is `Number*`, whereas the object being instantiated is type `Complex*`. Why is this legal syntax?

[2 Marks]

(d) On line 34 of Listing 2 we see the following:

```
c2->display();
```

In Listing 2 there are two versions of `display()`. Which one is called and why?

[3 Marks]

Q3. Object Orientated Programming

(a) Using examples of your choice, explain the concept of **function overloading** in C++.

[2 Marks]

(b) Briefly explain what is meant by **composition** in object orientated programming, and give one disadvantage is has compared to class inheritance.

[3 Marks]

(Over/...)

Q4. Specialist FIFO data structure

The code in Listing 3 builds and runs correctly. The class `CustomSum` is is a specialist FIFO type that also performs custom processing on each element that is added to it. An example of how it is used is shown in the `main` function.

Listing 3: Listing for Section A, Q4

```
1  #include <iostream>
2  #include <functional>
3  using namespace std;
4
5  template<class T, int N>
6  class CustomSum {
7      private:
8      int idx=0;
9      T sum=(T)0;
10     T data[N];
11     function<T(T)>updateFunc;
12
13     public:
14     CustomSum& operator << (T sample) {
15         T newValue = updateFunc(sample);
16         idx = (idx+1)%N;
17         T oldest = data[idx];       //Copy oldest sample
18         data[idx] = newValue;       //Overwrite oldest sample
19         sum = sum + newValue - oldest; //Update sum
20         return *this;
21     }
22     operator T() {
23         return sum;
24     }
25     CustomSum( function<T(T)>fn ) : updateFunc(fn) {
26         for (int n=0; n<N; n++) data[n]=(T)0;
27     }
28 };
29
30 int main() {
31     auto fn = [](int u)  {
32         return u*u;
33     };
34     CustomSum<int,4> meanSquare(fn);
35     meanSquare << 1 << 3 << 2 << 4 << 2;
36     cout << meanSquare << endl;
37     return 0;
38 }
```

(Over/...)

(a) On line 34 in Listing 3, the object `fn` is passed by parameter. What type of object is `fn` and briefly explain how it is used to customise the behaviour of the `CustomSum` class.

[2 Marks]

(b) The type `CustomSum` in Listing 3 is said to be a template class. Briefly explain what this means, and how in this case it makes the class more reusable.

[3 Marks]

Q5. Blocking and Polling

(a) In real-time software, you sometimes hear the term blocking. Explain what this means and how it can be a problem for real-time software.

[3 Marks]

(b) Multitasking is used to describe software that seems to perform more than one task at a time. This might be to monitor multiple input devices and/or send data to multiple output devices. Using examples of your choice and/or a flow chart, briefly explain what is meant by a **rapid polling loop** and how it can be used for multi-tasking.

[3 Marks]

(c) Briefly explain why rapid-polling is not suitable for applications that need to perform accurate sampling (measuring an input at regular time intervals).

[2 Marks]

(Over/...)

Q6. Interrupts The code below in Listing 4 uses three interrupts. This is using the Mbed OS drivers we used throughout the module. Read through this code and answer the questions below.

Listing 4: Listing for Section A, Q6

```
using namespace uop_msb;
using namespace chrono;

volatile int counter=0;
InterruptIn btnA(BTN1_PIN);
InterruptIn btnB(BTN2_PIN);
Ticker tick;
DigitalOut redLED(TRAF_RED1_PIN);
DigitalOut yellowLED(TRAF_YEL1_PIN);
DigitalOut greenLED(TRAF_GRN1_PIN);

void funcA() {
    redLED = !redLED;
    counter++;
}

void funcB() {
    yellowLED = !yellowLED;
    counter++;
}

void funcTmr() {
    greenLED = !greenLED;
    counter++;
}

int main() {
    btnA.rise(&funcA);
    btnB.fall(&funcB);
    tick.attach(&funcTmr, 500ms);
    while (true) {
        sleep();
        printf("Number of interrupts: %d\n", counter);
    }
}
```

(a) Explain what needs to occur in order for the functions `funcA` and `funcB` to ever be called.

[2 Marks]

(Over/...)

(b) The engineer that wrote the code in Listing 4 says that the green LED should flash on and off once a second. Do you agree? Justify your answer.

[2 Marks]

(c) In the main function in Listing 4, there is a `printf` statement. Will this line ever be executed? Justify your answer.

[2 Marks]

(d) All the interrupt service routines in Listing 4 modify the same shared variable `counter`. This variable has been declared as `volatile`. What is the purpose of the `volatile` keyword?

[2 Marks]

(e) The engineer that wrote the code in Listing 4 claims that there are no race conditions in the code because the variable `counter` is declared as volatile. Do you agree? Justify your answer.

[3 Marks]

(Over/...)

Q7. Threads

Read through the code below in Listing 5 and answer the questions that follow.

Listing 5: Listing for Section A, Q7

```
#include "mbed.h"
#include "uop_msb.h"
using namespace uop_msb;

class FlashingLED {
    private:
    DigitalOut led;
    Thread t;

    private:
    void flash() {
        while(true) {
            led = !led;
            ThisThread::sleep_for(500ms);
        }
    }

    public:
    FlashingLED(PinName pin) : led(pin) {
        t.start(callback(this, &FlashingLED::flash));
    }
};
void task1() {
    DigitalOut red_led(TRAF_RED1_PIN);
    DigitalIn sw1(BTN1_PIN);
    red_led = sw1;
    while(true) {
        while (sw1 == 0) {};
        ThisThread::sleep_for(50ms);
        while (sw1 == 1) {};
        red_led = !red_led;
        ThisThread::sleep_for(50ms);
    }
}

int main() {
    Thread t1;
    FlashingLED yellow(TRAF_YEL1_PIN);
    t1.start(task1);
    t1.join();
}
```

(Over/...)

The following questions all refer to code Listing 5.

- The pin label `TRAF_YEL1_PIN` refers to a pin connected to a yellow LED.
- The pin label `TRAF_RED1_PIN` refers to a pin connected to a red LED.

(a) When this code is run, does the yellow LED ever flash? Justify your answer.
[3 Marks]

(b) The function `task1` is run in a thread. Consider what would happen if main was changed as shown below.

```
int main() {
   FlashingLED yellow(TRAF_YEL1_PIN);
   task1();
}
```

Would this affect the behaviour of the application? Justify your answer.
[3 Marks]

END OF SECTION A

(Over/...)

SECTION B: Answer TWO questions.

Q8. Question B.8

Carefully read through the code in Listing 6. The code has been written with Mbed OS to regularly sample an ADC and write the data to the serial terminal. LEDs are also used as output devices. Two buttons are used as input devices.

(a) When no buttons are pressed:

(i) Which LEDs flash and at what rate?

[2 Marks]

(ii) Is any data written to the serial terminal? Justify your answer.

[2 Marks]

(b) The following line blocks in the **WAITING** state. What causes this to unblock and when?

```
ThisThread::flags_wait_any(SAMP_SIGNAL);
```

[4 Marks]

(c) If the blue button is held down, the green LED stops flashing.

(i) Why is this? Justify your answer

[3 Marks]

(ii) Will samples be lost? Justify your answer

[3 Marks]

(d) Now consider what happens if button A is held down:

(i) Will any LEDs stop flashing, and if so, which? Justify your answer

[2 Marks]

(ii) Will data be lost? Justify your answer

[4 Marks]

(e) This code makes use of closures. Explain the concept of a closure and what is meant by "capturing behaviour".

[5 Marks]

(Over/...)

Listing 6: Listing for Section B, Q8

```
const uint32_t SAMP_SIGNAL = 1;
DigitalOut greenLed(LED1);
DigitalOut blueLed(LED2);
DigitalIn  blueButton(USER_BUTTON);
DigitalIn  buttonA(BTN1_PIN);
AnalogIn   adc(AN_POT_PIN);

int main() {
    // ******** Logging Queue **********
    EventQueue logQueue;
    Thread tLogging(osPriorityBelowNormal);
    auto logging = [&logQueue]() {
        logQueue.dispatch_forever();
    };
    tLogging.start(logging);

    // ******** Sampling Thread **********
    auto sampLoop = [&logQueue]() {
        while(true) {
            ThisThread::flags_wait_any(SAMP_SIGNAL);
            float samp = adc;
            blueLed= !blueLed;
            logQueue.call(printf, "%5.3f ", samp);
            while (blueButton == 1);
        }
    };
    Thread tSamp(osPriorityHigh);
    tSamp.start(sampLoop);

    //Ticker ISR
    auto isr = [&tSamp]() {
        tSamp.flags_set(SAMP_SIGNAL);
    };
    Ticker sampleTimer;
    sampleTimer.attach(isr, 100ms);

    // ******** Main Loop **********
    while (true) {
        greenLed = !greenLed;
        ThisThread::sleep_for(500ms);
        while (buttonA == 1);
    }
} //end main
```

(Over/...)

Q9. Question B.9

Consider a real-time system, with embedded software written with Mbed, with the real-time scheduler enabled.

(a) Very often when a microcontroller has to interface to the real world, it must interact with blocking hardware that is slower than the CPU. Describe **TWO** significant problems this can present when writing embedded software. Give examples to illustrate your answer.

[4 Marks]

(b) Two methods for managing real-time interfacing in embedded software are interrupts and multi threaded programming.

(i) Describe what is meant by multi threaded programming and explain how it helps to write software that interacts with multiple devices.

[4 Marks]

(ii) Describe what is meant by hardware interrupts and explain how it helps to write software that interacts with multiple devices.

[4 Marks]

(iii) Describe one of the limitations of using interrupts for multi-tasking.

[2 Marks]

(iv) One of the dangers of using threads is to accidentally create a deadlock. With the aid of an example, briefly explain what is meant by a deadlock.

[4 Marks]

(c) Consider the code in Listing 7. The code updates the variable counter when buttons are pressed. It also displays the value of the counter in the serial terminal.

(i) The developer has used the statement `CriticalSectionLock lock` in both interrupt service routines. What does this do and why do you think it was included?

[3 Marks]

(ii) The developer has tried to avoid a race condition by using a Mutex lock in the `flashYellow()` function. Is this effective? Justify your answer.

[4 Marks]

(Over/...)

Listing 7: Listing for Section B, Q9

```
Mutex counterLock;
volatile uint32_t counter=0;
Thread t1;

InterruptIn btnA(BTN1_PIN), btnB(BTN2_PIN);
DigitalOut redLED(TRAF_RED1_PIN), yellowLED(TRAF_YEL1_PIN), greenLED(TRAF_GRN1_PIN);

void funcA() {
    btnA.rise(NULL);
    redLED = !redLED;
    wait_us(100000);     //Delay
    btnA.rise(&funcA);
    CriticalSectionLock lock;
    counter = counter + 1;
    t1.flags_set(1);
}

void funcB() {
    btnB.rise(NULL);
    greenLED = !greenLED;
    wait_us(100000);     //Delay
    btnB.rise(&funcB);
    CriticalSectionLock lock;
    counter = counter - 1;
    t1.flags_set(2);
}


void flashYellow() {
    while (true) {
        ThisThread::flags_wait_any(1 | 2);
        yellowLED = !yellowLED;
        counterLock.lock();
        printf("%u\n", counter);
        counterLock.unlock();
    }
}

int main()  {
    btnA.rise(&funcA);  btnB.rise(&funcB);
    t1.start(flashYellow);
    t1.join();
}
```

(Over/...)

Q10. Question B.10

Consider a real-time system, with embedded software written with Mbed OS, with the real-time scheduler enabled. The code in Listing 8 is an attempt to write a First In First Out (FIFO) buffer.

(a) The class FIFO has two members of type `Semaphore`. Explain the function of a Semaphore and it's role in this class.

[6 Marks]

(b) This class claims to conform to the producer-consumer pattern. Do you agree? Justify your answer.

[2 Marks]

(c) An instance of the FIFO class is created as follows:

```
FIFO<float, 8> fifo8;
```

(i) If the function `getSample(0.0f)` is called 9 times on a dedicated thread, it is found to block. Is this correct and desirable behaviour? Justify your answer.

[4 Marks]

(ii) What conditions would unblock the thread in (i)? Justify your answer.

[3 Marks]

(d) This class is a template class. Briefly explain what this means and describe one of the benefits of using templates in embedded software.

[4 Marks]

(e) FIFO buffering is often used in real-time software. A statement often given is: "To avoid data loss, the average rate of consumption must be less than the average rate of production". Explain what is meant by this, and what assumptions (if any) are being made.

[6 Marks]

(Over/...)

Listing 8: Listing for Section B, Q10

```
template <class T, int N>
class FIFO {
    private:
    T buffer[N];
    Semaphore samplesInBuffer, spaceInBuffer;
    uint32_t head=0, tail=0;
    Mutex bufferLock;

    public:
    FIFO() : samplesInBuffer(0), spaceInBuffer(N) {
        for (uint32_t n=0; n<N; n++) buffer[n] = (T)0;
    }

    void addSample(T u) {
        spaceInBuffer.acquire();
        bufferLock.lock();
        buffer[head] = u;
        head = (head + 1) % N;
        bufferLock.unlock();
        samplesInBuffer.release();
    }

    T getSample() {
        T y;
        samplesInBuffer.acquire();
        bufferLock.lock();
        y = buffer[tail];
        tail = (tail + 1) % N;
        bufferLock.unlock();
        spaceInBuffer.release();
        return y;
    }
};
```

(Over/...)

END OF QUESTIONS

END OF PAPER